

LAB MANUAL

Experiment Title: GUI Development using Java Swing

1. Aim

To design and implement GUI-based applications using Java Swing and understand event-driven programming.

2. Objectives

- To understand basic GUI components
 - To implement event handling using ActionListener
 - To build interactive applications using Swing
 - To apply object-oriented concepts in GUI design
-

3. Theory

Java Swing is used to build platform-independent graphical user interfaces. It follows **event-driven programming**, where user actions (events) trigger responses.

Key Concepts

- **Component:** UI element (JButton, JLabel)
 - **Container:** Holds components (JFrame, JPanel)
 - **Event:** User action (click, input)
 - **Listener:** Handles events
-

4. Pre-Lab Questions

1. What is GUI?
 2. What is a component in Swing?
 3. What is event-driven programming?
 4. What is JFrame used for?
 5. What is an event listener?
-

PART A: BEGINNER EXPERIMENTS

Experiment 1: Hello GUI

Program

```
import javax.swing.*;

public class HelloGUI {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Hello");

        JLabel label = new JLabel("Hello, Swing!");

        frame.add(label);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Experiment 2: Button Click Message

Program

```
import javax.swing.*;
import java.awt.*;

public class ButtonClick {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Button Example");
        frame.setLayout(new FlowLayout());

        JButton button = new JButton("Click Me");

        button.addActionListener(e -> {
            JOptionPane.showMessageDialog(frame, "Button Clicked");
        });

        frame.add(button);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

```
}  
}
```

Experiment 3: Text Input Display

Program

```
import javax.swing.*;  
import java.awt.*;  
  
public class InputDisplay {  
    public static void main(String[] args) {  
  
        JFrame frame = new JFrame("Input Example");  
        frame.setLayout(new FlowLayout());  
  
        JTextField input = new JTextField(10);  
        JButton button = new JButton("Show");  
        JLabel label = new JLabel();  
  
        button.addActionListener(e -> {  
            label.setText("You entered: " + input.getText());  
        });  
  
        frame.add(input);  
        frame.add(button);  
        frame.add(label);  
  
        frame.setSize(300, 150);  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setVisible(true);  
    }  
}
```

PART B: INTERMEDIATE EXPERIMENTS

Experiment 4: Simple Calculator

Program

```
import javax.swing.*;
import java.awt.*;

public class Calculator {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Calculator");
        frame.setLayout(new FlowLayout());

        JTextField num1 = new JTextField(5);
        JTextField num2 = new JTextField(5);
        JButton add = new JButton("Add");
        JLabel result = new JLabel("Result:");

        add.addActionListener(e -> {
            try {
                int a = Integer.parseInt(num1.getText());
                int b = Integer.parseInt(num2.getText());
                result.setText("Result: " + (a + b));
            } catch (Exception ex) {
                result.setText("Invalid Input");
            }
        });

        frame.add(num1);
        frame.add(num2);
        frame.add(add);
        frame.add(result);

        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Experiment 5: To-Do List

Program

```
import javax.swing.*;
import java.awt.*;

public class TodoApp {
    public static void main(String[] args) {

        JFrame frame = new JFrame("To-Do List");
        frame.setLayout(new FlowLayout());

        JTextField input = new JTextField(10);
        JButton add = new JButton("Add");
        JButton remove = new JButton("Remove");

        DefaultListModel<String> model = new DefaultListModel<>();
        JList<String> list = new JList<>(model);

        add.addActionListener(e -> {
            if (!input.getText().isEmpty()) {
                model.addElement(input.getText());
                input.setText("");
            }
        });

        remove.addActionListener(e -> {
            int index = list.getSelectedIndex();
            if (index != -1) {
                model.remove(index);
            }
        });

        frame.add(input);
        frame.add(add);
        frame.add(remove);
        frame.add(new JScrollPane(list));

        frame.setSize(300, 250);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

To do:

Scenario-Based GUI Development

Task 1: Mini Food Ordering Application

Problem Statement

Design a GUI-based food ordering system for a college canteen.

Requirements

- Display food items with prices:
 - Pizza (₹100)
 - Burger (₹80)
 - Juice (₹50)
- Allow users to select multiple items
- Provide a button: “Calculate Bill”
- Display:
 - Selected items
 - Total amount

Functional Expectations

- If no item is selected, display an appropriate message
- Output should be clearly formatted

Code:

```
import javax.swing.*;
import java.awt.*;

public class FoodOrderApp extends JFrame {

    // Checkboxes for items
    private JCheckBox pizza, burger, juice;

    // Output area
    private JTextArea output;
```

```

public FoodOrderApp() {

    setTitle("Food Ordering App");
    setSize(350, 300);
    setLayout(new FlowLayout());
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // Create checkboxes with prices
    pizza = new JCheckBox("Pizza - ₹100");
    burger = new JCheckBox("Burger - ₹80");
    juice = new JCheckBox("Juice - ₹50");

    // Button
    JButton billButton = new JButton("Calculate Bill");

    // Output area
    output = new JTextArea(8, 25);
    output.setEditable(false);

    // Add components
    add(new JLabel("Select Items:"));
    add(pizza);
    add(burger);
    add(juice);
    add(billButton);
    add(new JScrollPane(output));

    // Event handling
    billButton.addActionListener(e -> calculateBill());

    setVisible(true);
}

// Method to calculate total and display result
private void calculateBill() {

    int total = 0;
    StringBuilder bill = new StringBuilder("Selected Items:\n");

    if (pizza.isSelected()) {
        total += 100;
        bill.append("Pizza - ₹100\n");
    }

    if (burger.isSelected()) {
        total += 80;
        bill.append("Burger - ₹80\n");
    }

    if (juice.isSelected()) {

```

```

        total += 50;
        bill.append("Juice - ₹50\n");
    }

    bill.append("\nTotal: ₹").append(total);

    // Display in text area
    output.setText(bill.toString());
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new FoodOrderApp());
}
}

```

Task 2: Student Grade Evaluator

Problem Statement

Design a GUI application to evaluate a student's academic performance.

Requirements

- Input marks for 3 subjects
- Provide a button: "Evaluate"
- Display:
 - Total marks
 - Average marks
 - Grade

Grading Criteria

- $\geq 75 \rightarrow$ Grade A
- $\geq 60 \rightarrow$ Grade B
- $\geq 40 \rightarrow$ Grade C
- $< 40 \rightarrow$ Fail

Functional Expectations

- Handle invalid inputs (non-numeric values)
- Display results clearly

5. Post-Lab Questions

1. What is the role of JFrame?
2. What is an event in GUI?
3. How does ActionListener work?
4. Why is exception handling required in GUI input?
5. What is the role of Timer in Swing?